

Python Basics

This module introduces the building blocks of every Python program: values and variables, decisions and loops, and functions. Everything else in the course builds on these three ideas.

Variables and Data Types

Prerequisites: none. This is the first topic of the course.

A variable is a name that refers to a value. In Python you create a variable simply by assigning to it, for example `age = 21`. Python is dynamically typed: the type belongs to the value, not to the variable, so the same name can later refer to a value of a different type.

The core built-in types are:

- `int` for whole numbers such as 42
- `float` for decimal numbers such as 3.14
- `str` for text such as "hello"
- `bool` for the two truth values `True` and `False`
- `None`, a special value meaning "nothing here"

Use the built-in function `type()` to see the type of a value. Converting between types is called casting: `int("7")` gives 7 and `str(7)` gives "7". A common beginner mistake is adding a string and a number, which raises a `TypeError`, because Python never converts types silently in that case.

Control Flow

Prerequisites: Variables and Data Types.

Control flow decides which lines run and how many times. An `if` statement runs a block only when a condition is `True`; `elif` and `else` handle the other cases. Python uses indentation (four spaces by convention) instead of braces to mark blocks, so wrong indentation is a real error, not just bad style.

Loops repeat work. A `for` loop iterates over the items of a sequence, for example `for name in names:`. The `range(n)` function produces the numbers 0 to `n-1`, which is why `for i in range(3)` runs three times. A `while` loop repeats as long as its condition stays `True`, so you must make sure the condition eventually becomes `False` or the loop never ends.

Inside loops, `break` exits the loop immediately and `continue` skips to the next iteration. Comparison operators (`==`, `!=`, `<`, `>`) and logical operators (`and`, `or`, `not`) build the conditions that control flow depends on.

Functions

Prerequisites: Variables and Data Types, Control Flow.

A function is a named, reusable block of code defined with `def`. It takes parameters, does some work and usually returns a value with `return`. A function without a return statement returns `None`.

Functions make programs easier to read and test because each function does one job. Parameters can have default values, as in `def greet(name, greeting="Hello")`, and callers can pass arguments by name, which is called keyword arguments.

Variables created inside a function are local: they exist only while the function runs. This is called scope. Reading a global variable inside a function works, but changing it requires the `global` keyword, which is usually a sign of poor design.

Functions are values too, so you can pass a function to another function. A short anonymous function can be written with `lambda`, for example `lambda x: x * 2`. Later topics such as recursion in the DSA course rely on a solid understanding of how functions call other functions.